AGM NoSQL Database Use Cases

Ana Zapata, David Sherman, Janhavi Subedarpage

Agenda

- Goal & NoSQL Introduction
- NoSQL Database Use Cases For AGM
  - MongoDB
  - Redis
  - Neo4j
- Questions

Overview

- Goal: To coordinate efforts between AGM's Data Engineering & Data Science teams to enhance & streamline daily operational processes utilizing NoSQL databases

What is NoSQL?

- "Non-SQL", an approach to database design that enables data storage and querying outside of traditional relational database structures

Category	SQL	NoSQL
Type	Relational database management system (RDBMS)	Non-relational system
Language	Fixed, dynamic, or predefined schema	Dynamic schema
Storage Type	Adequate for hierarchical data storage	Inadequate for hierarchical data storage
Query Complexity	Adequate for complex scripts	Inadequate for complex scripts
Scalability	Vertical	Horizontal
Examples	MySQL, Microsoft SQL Server, PostgreSQL	MongoDB, Redis, Neo4j

AGM NoSQL Use Cases

AGM Use Case: Traffic

- AGM prides itself on its ability to deliver fresh meals to customers, though tough traffic conditions can delay delivery times
  - This can lead to food spoilage on the way to the customer, therefore ruining the customer's experience and risk him/her not wanting to use AGM anymore
- Most roads are close due to impending holidays, events, or horrific traffic jams
  - How can we deliver data about these upcoming delays to delivery drivers?

Traffic Solution: Use MongoDB

- Use MongoDB, a document-oriented database management system, to store live traffic issues that are known in advance, such as construction, special events, and holidays
- Break the area into a grid - when a vehicle has a route planned, we only have to pull grid pieces that are in potential routes and not all traffic
  - This can be performed by pulling details about the particular road, with latitude and longitude, that the driver has to navigate
- Why MongoDB?
  - Document-style model that allows for quick JSON retrieval, parsing and easier real-time updates
  - Scaled out architecture will be able to handle spikes in traffic as AGM grows (more routes to assess)
  - Diverse data (i.e. various traffic issues) are best supported by this NoSQL database
  - More dynamic options for enhancing the application over time
- Why RDBMS may not be a good fit?
  - If traffic data is stored across multiple tables (i.e. roads with traffic issues, routes those roads serve), it'd be more time-consuming to deliver updates to drivers

AGM Use Case: Delivery Truck Info

- To provide AGM and customers insights into the status of the food they ordered and the trucks that are delivering the orders, it'd be useful to know real-time information on trucks, such as:
  - Location: where on the delivery route is the driver?
  - Fuel Status: will the driver need more fuel either before or along the route?
- Not knowing this information could make customers impatient and slow down delivery times if the driver has to stop for gas or a recharge

Delivery Truck Info Solution: Use Redis

- Use Redis, an in-memory database management system, to provide real-time truck locations and provide recommendations on when to refuel based on planned routes
  - Through an AGM mobile application, store managers and customers can view on a map where the truck is located along with the ETA
  - For drivers, the app will determine, based on current fuel & burn rate, how many more stops can the driver make prior to a refuel
- Why Redis?
  - Fast read & write operations allow for quick data retrieval (i.e. calculating stops, ETA)
  - Can support high levels of concurrent processes & easily integratable with other tools
  - Horizontal scalability allows for new nodes, or stops, to constantly be added
- Why RDBMS may not be a good fit?
  - Not as adept at storing and retrieving real-time data as Redis
  - Vertically scaling a single SQL database instance is more costly if nodes to be added

AGM Use Case: BART Integrations

- For customers who want to pick up their order, how can AGM provide the quickest route, factoring in real-time train delays?
- How can AGM identify stations with the most off-campus students?
  - Will serve as sample population for this use case due to proximity to Berkeley AGM store (the test location for this POC)
- How can AGM determine which are the busiest BART stations based on incoming and outgoing routes from that specific station?

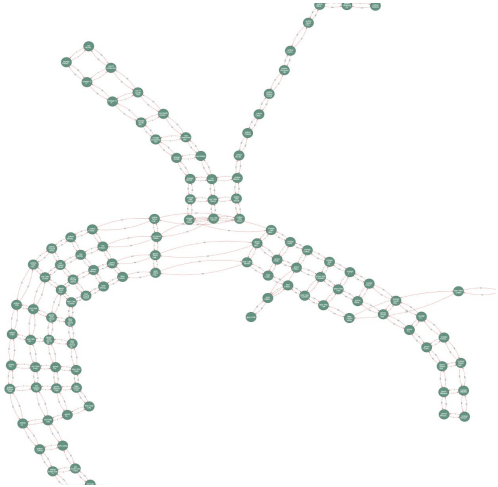
BART Integrations Solution: Use Neo4j

- Use Neo4j, a NoSQL graph database management system, to identify trends by neighborhood in real-time
- By integrating the BART API into a mobile app with Neo4j, AGM and its customers can view:
  - A real-time map of the shortest possible path and time to the store of interest
  - Top college student neighborhoods (based on info from Berkeley's off-campus website)
  - High traffic BART stations that will enable AGM to view sales trends by region
- Why Neo4j?
  - Most suitable DBMS to represent geographical relationships
  - Adept at representing semi-structured data
  - Does not require complex JOINS to connect different nodes in datasets
- Why RDBMS may not be a good fit?
  - More of an inherent tabular data management system rather than a graph-based management system, which is what we'd want for plotting data on maps

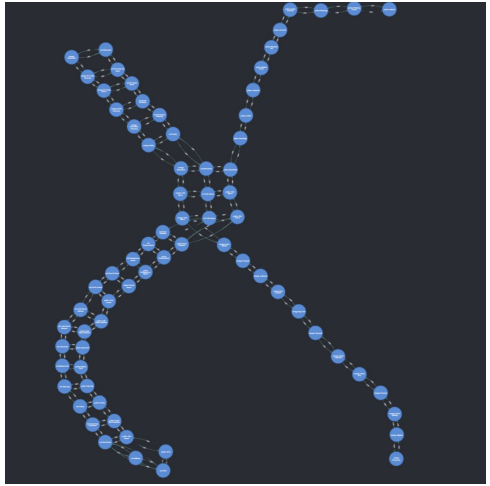
BART Map Visuals

BART Map From www.BART.gov

BART Map - Neo4j (All Lines)

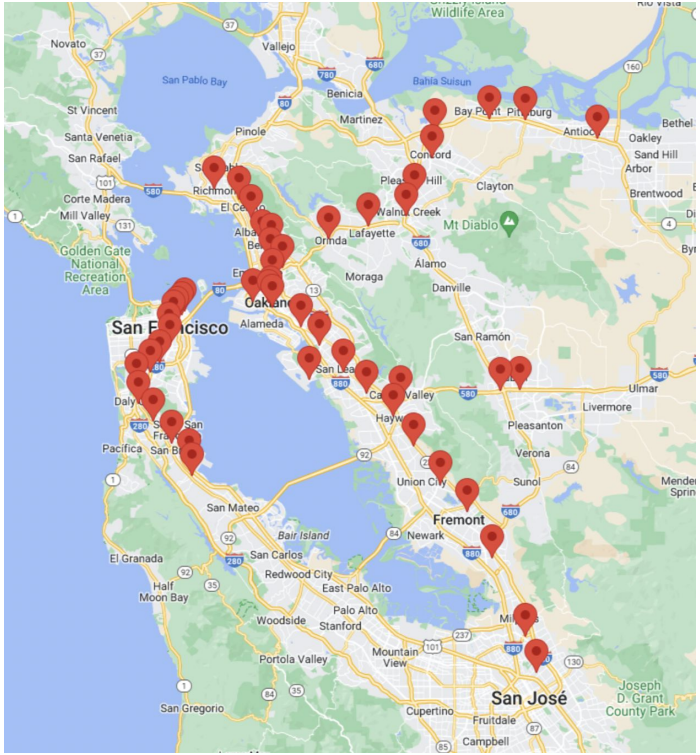


BART Map - Neo4j (Orange, Yellow, Red Lines)



BART Map Visuals

BART Station Locations - Google Maps



BART Shortest Possible Paths Algorithm

Depart from Downtown Berkeley, arrive Rockridge

Minutes: 12.0

Depart from Downtown Berkeley, arrive North Berkeley

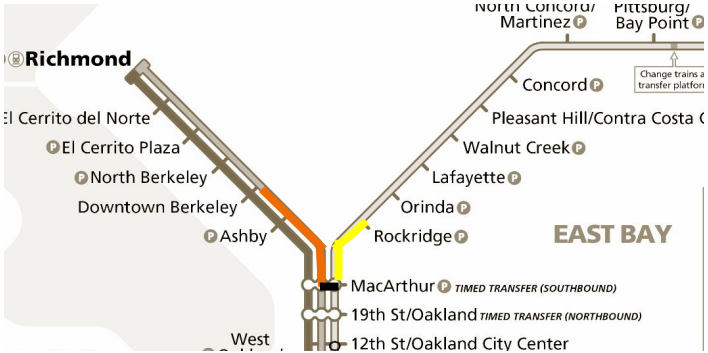
Minutes: 2.0

Depart from Downtown Berkeley, arrive El Cerrito Plaza

Minutes: 5.0

Depart from Downtown Berkeley, arrive lake Merritt

Minutes: 15.0




```
In [90]: my_neo4j_shortest_path('depart Downtown Berkeley', 'arrive Rockridge')

--------------------------------
   Total Cost:  719
   Minutes:  12.0
--------------------------------
depart Downtown Berkeley, 0, 0
orange Downtown Berkeley, 0, 0
orange Ashby, 180, 180
orange MacArthur, 240, 420
yellow MacArthur, 59, 479
yellow Rockridge, 240, 719
arrive Rockridge, 0, 719
```

```
In [91]: my_neo4j_shortest_path('depart Downtown Berkeley', 'arrive Lake Merritt')

--------------------------------
   Total Cost:  900
   Minutes:  15.0
--------------------------------
depart Downtown Berkeley, 0, 0
orange Downtown Berkeley, 0, 0
orange Ashby, 180, 180
orange MacArthur, 240, 420
orange 19th Street, 180, 600
orange 12th Street, 120, 720
orange Lake Merritt, 180, 900
arrive Lake Merritt, 0, 900
```

API Integration

In the future we will include the real time estimation provided by the BART API to our shortest path algorithm.

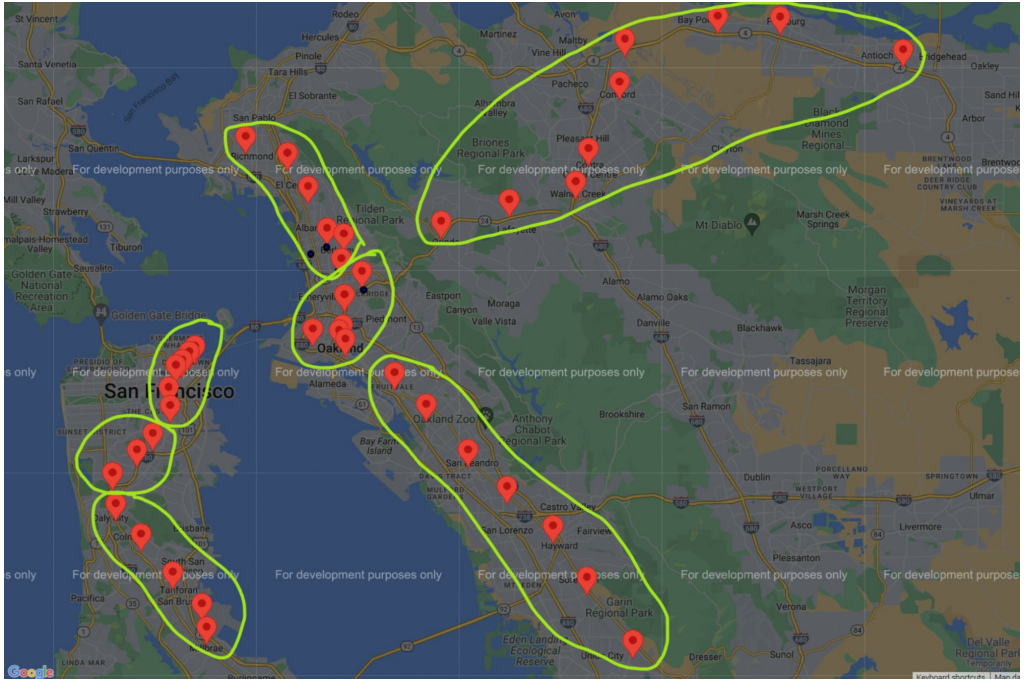
Red line

station_name	destination	minutes	platform	direction
Downtown Berkeley	Berryessa	15	2	South
Downtown Berkeley	Berryessa	45	2	South
Downtown Berkeley	Richmond	7	1	North

Orange line

station_name	destination	minutes	platform	direction
Downtown Berkeley	Millbrae/SFO	33	2	South
Downtown Berkeley	Millbrae/SFO	58	2	South
Downtown Berkeley	Richmond	4	1	North
Downtown Berkeley	Richmond	17	1	North

Community Detection Algorithm



	name	community
0	orange 12th Street	5.0
1	red 12th Street	5.0
2	yellow 12th Street	5.0
3	orange 19th Street	5.0
4	red 19th Street	5.0
5	yellow 19th Street	5.0
6	orange MacArthur	5.0
7	red MacArthur	5.0
8	yellow MacArthur	5.0
9	yellow Rockridge	5.0

	name	community
63	yellow Antioch	68.0
64	yellow Concord	68.0
65	yellow Lafayette	68.0
66	yellow North Concord	68.0
67	yellow Orinda	68.0
68	yellow Pittsburg	68.0
69	yellow Pittsburg Center	68.0
70	yellow Pleasant Hill	68.0
71	yellow Walnut Creek	68.0

Betweenness Centrality Algorithm

In [84]:
```
query = """

CALL gds.betweenness.stream('bart_map')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score as betweenness
ORDER BY betweenness DESC

"""



my_neo4j_run_query_pandas(query)
```

Out[84]:

	name	betweenness
0	green West Oakland	3142.520859
1	green Lake Merritt	3066.190791
2	green Fruitvale	2771.667481
3	red West Oakland	2726.483864
4	red 12th Street	2658.950211
...	...	...
109	yellow SFO	6.055576
110	gray OAK	0.000000
111	yellow Antioch	0.000000
112	blue Dublin	0.000000
113	red Millbrae	0.000000

114 rows × 2 columns

Questions?

Sources

- NoSQL Databases:
  - https://www.geeksforgeeks.org/difference-between-sql-and-nosql/
  - https://www.ibm.com/topics/nosql-databases
  - https://www.mongodb.com/why-use-mongodb
  - https://www.mongodb.com/compare/mongodb-mysql
  - https://levelup.gitconnected.com/redis-vs-other-databases-an-in-depth-comparison-of-sql-and-nosql-solutions-7c4a9ca9183
  - https://www.tutorialspoint.com/neo4j/neo4j_features_advantages.htm
- UC Berkeley Off-Campus Top Neighborhoods & Demographics:
  - https://www.law.berkeley.edu/housing/neighborhood-guide/
  - https://www.niche.com/places-to-live/n/north-berkeley-berkeley-ca/residents/
  - https://www.niche.com/places-to-live/n/south-berkeley-berkeley-ca/residents/
  - https://www.niche.com/places-to-live/n/west-berkeley-south-berkeley-ca/residents/
  - https://www.niche.com/places-to-live/albany-alameda-ca/residents/
  - https://www.niche.com/places-to-live/el-cerrito-contra-costa-ca/residents/
  - https://www.niche.com/places-to-live/n/rockridge-oakland-ca/residents/
  - https://www.niche.com/places-to-live/n/merritt-oakland-ca/residents/

What is NoSQL?

- “Non-SQL”, an approach to database design that enables data storage and querying outside of traditional relational database structures
- Advantages:
  - **Horizontal scalability** - can address large sets of dynamic data better than SQL
  - **Cost-effective** - no licenses, less resources needed to build out additional data
  - **Replication** - copes & stores data across multiple servers, good in case of downtime
  - **Speed** - faster processing speeds and can adapt to agile storage, optimal for web/mobile apps

BART Shortest Possible Paths Algorithm

```
my_neo4j_shortest_path('depart Downtown Berkeley', 'arrive Rockridge')
```

```
--------------------------------
   Total Cost:  719
   Minutes:  12.0
--------------------------------
depart Downtown Berkeley, 0, 0
orange Downtown Berkeley, 0, 0
orange Ashby, 180, 180
orange MacArthur, 240, 420
yellow MacArthur, 59, 479
yellow Rockridge, 240, 719
arrive Rockridge, 0, 719
```

```
my_neo4j_shortest_path('depart Downtown Berkeley', 'arrive Lake Merritt')
```

```
--------------------------------
   Total Cost:  900
   Minutes:  15.0
--------------------------------
depart Downtown Berkeley, 0, 0
orange Downtown Berkeley, 0, 0
orange Ashby, 180, 180
orange MacArthur, 240, 420
orange 19th Street, 180, 600
orange 12th Street, 120, 720
orange Lake Merritt, 180, 900
arrive Lake Merritt, 0, 900
```

```
my_neo4j_shortest_path('depart Downtown Berkeley', 'arrive Ashby')
```

```
--------------------------------
   Total Cost:  180
   Minutes:  3.0
--------------------------------
depart Downtown Berkeley, 0, 0
red Downtown Berkeley, 0, 0
red Ashby, 180, 180
arrive Ashby, 0, 180
```

```
my_neo4j_shortest_path('depart Downtown Berkeley', 'arrive North Berkeley')
```

```
--------------------------------
   Total Cost:  120
   Minutes:  2.0
--------------------------------
depart Downtown Berkeley, 0, 0
red Downtown Berkeley, 0, 0
red North Berkeley, 120, 120
arrive North Berkeley, 0, 120
```

```
my_neo4j_shortest_path('depart Downtown Berkeley', 'arrive El Cerrito Plaza')
```

```
--------------------------------
   Total Cost:  300
   Minutes:  5.0
--------------------------------
depart Downtown Berkeley, 0, 0
orange Downtown Berkeley, 0, 0
orange North Berkeley, 120, 120
orange El Cerrito Plaza, 180, 300
arrive El Cerrito Plaza, 0, 300
```

What is MongoDB?

- A NoSQL database management system that can manage document-oriented information, store or retrieve information
- Architecture is comprised of collections & documents:
  - Collections: key-value pairs, MongoDB's basic unit of data
  - Documents: equivalent to SQL tables and contain document sets
- Features:
  - Indexing: useful for efficient searching of documents
  - Replication & High Availability: mitigates risk of hardware failure
  - Aggregation Capabilities: easy to process multiple data records (i.e. sum, avg, min...)
  - High Speed: more efficient processing capabilities due to efficient indexing & storage methods